UNIVERSIDAD ARGENTINA DE LA EMPRESA

UADE

Algoritmos y Estructura de Datos 1 · Programación 1

TRABAJO PRÁCTICO INTEGRADOR · SEGUNDA PARTE

Room Master

Sistema de Gestión Hotelera



Documento explicativo de la ampliación y mejora del sistema, según la **Sección 15** de la consigna: mejoras, estructuras de datos, persistencia, excepciones, pruebas unitarias y cambios respecto de la primera parte.

DOCENTE	Lic. Gustavo E. Escandell
INTEGRANTES	Menendez Mojana, Tiara (referente) Ramos, Lucas Sebastián Iglesias, Luca Rueda, Leandro Nicolás
CUATRIMESTRE	2.º cuatrimestre · 2026
ENTREGA	05/11/2026
REPOSITORIO	github.com/lucassebastianramos/room-master

Índice

1 Resumen del sistema y mapa de requisitos — Secciones 2 y 14

2 Descripción de las mejoras — Ítem 15.a

3 Estructuras de datos — Ítem 15.b · Sección 5

4 Persistencia — Ítem 15.c · Secciones 7 y 8

5 Excepciones — Ítem 15.d · Sección 6

6 Pruebas unitarias — Ítem 15.e · Sección 9

7 Cambios respecto de la Parte 1 — Ítem 15.f · Sección 3

8 Cumplimiento de las restricciones técnicas — Sección 12

A Anexo A · Arquitectura y módulos — Sección 10

B Anexo B · Ciclo de vida de una habitación — Sección 13

C Anexo C · Instrucciones de ejecución en otro equipo — Sección 16

> **Cómo leer este documento**
> Cada capítulo responde un ítem de la Sección 15 en el mismo orden que la consigna. Los nombres en `monoespaciado` son funciones o archivos reales del proyecto, para poder ubicarlos en el código durante la defensa. Las referencias del tipo *Sección 9* remiten al enunciado de la Segunda Parte.

Convenciones

Marca	Significado
NUEVA	Funcionalidad que no existía en la Parte 1.
MEJORADA	Funcionalidad de la Parte 1 que se reescribió o amplió.
CORREGIDA	Error o comportamiento indebido detectado en la Parte 1 y solucionado.
IMPLEMENTADO	Disponible en el sistema y cubierto por la documentación.

1. Resumen del sistema

SECCIONES 2 Y 14

Room Master resuelve la operación de la recepción de un hotel: conocer el estado de cada habitación (libre, ocupada o en limpieza), registrar ingresos y salidas, reubicar huéspedes, cobrar según el piso y la duración de la estadía, y consultar indicadores de ocupación y recaudación. En esta segunda parte el sistema pasó de un único archivo con datos en memoria a una aplicación modular que **conserva la información entre ejecuciones**.

7	45	10/10	0
módulos	funciones del sistema	pruebas aprobadas	clases · bases de datos

Mapa de requisitos obligatorios (Sección 14)

Requisito	Cómo se cumple	Dónde	Estado
Continuar el proyecto de la Parte 1	Misma problemática (hotel) y mismas operaciones base, evolucionadas.	Todo el proyecto	CUMPLE
Mejorar funcionalidades existentes	Huésped como diccionario, swap y tarifas divididos en subfunciones, cancelación con `-1`.	`huespedes.py`, `tarifas.py`	CUMPLE
Incorporar nuevas funcionalidades	Búsqueda por nombre y por habitación, reporte de libres, DNI único, guardado y recuperación.	`huespedes.py`, `reportes.py`, `archivos.py`	CUMPLE
Conjuntos	Unicidad de DNI y cálculo de habitaciones libres/ocupadas.	`validar_dni`, `reporte_habitaciones_libres`	CUMPLE
Tuplas	Identidad `(dni, nombre)` y retornos múltiples de coordenadas.	`realizar_checkin`, `hotel.py`	CUMPLE
Diccionarios	Registro de huésped y estructura del archivo JSON.	`realizar_checkin`, `guardar_datos_hotel`	CUMPLE
Excepciones	`try / except / else / finally` en lectura y escritura de archivos.	`archivos.py`	CUMPLE
Archivos JSON	Guardado después de cada operación y recuperación al iniciar.	`archivos.py`, `main.py`	CUMPLE
Archivos de texto	Bitácora de check-outs (escritura `"a"`, lectura y exportación CSV).	`facturacion.txt`	CUMPLE
Pruebas unitarias	10 pruebas sobre 6 funciones, con la distribución 4 / 3 / 2 / 1.	`tests.py`	CUMPLE
Validaciones	Regex, rangos, estado del sistema y reintento ante error.	`validaciones.py`	CUMPLE
Modularización	7 módulos; `main.py` solo coordina; sin variables globales.	Anexo A	CUMPLE
Clases · Bases de datos · Librerías no vistas	No se utilizan.	Capítulo 8	CUMPLE

Integración de los contenidos (Sección 13)

Los conceptos no aparecen aislados. Una sola operación, el **check-in**, atraviesa casi todos:

Validación regex de nombre, DNI y días `validar_generico` → **Conjunto** DNI no repetido `validar_dni` → **Tupla + dict** registro del huésped `realizar_checkin` → **JSON + excepciones** guardado seguro `guardar_datos_hotel` → **Recuperación** al reiniciar `cargar_datos_hotel` → **Prueba** búsqueda del huésped `test_07`, `test_10`

2. Descripción de las mejoras

ÍTEM 15.A

Funcionalidades modificadas o incorporadas respecto de la primera entrega. La comparación se hizo contra la última versión de la Parte 1 en el repositorio (commit `495246d`, 13/09/2026), que constaba de un único archivo `main.py`.

2.1 Funcionalidades

Funcionalidad	Tipo	Qué cambió y por qué
Guardado y recuperación	NUEVA	Antes, al cerrar el programa se perdía todo. Ahora el estado se guarda en `hotel_data.json` después de cada operación y se recupera al iniciar.
DNI único	NUEVA	La Parte 1 permitía registrar dos veces el mismo DNI. `validar_dni` lo impide usando un conjunto y pide otro DNI.
Búsqueda por nombre	NUEVA	Coincidencia parcial sin distinguir mayúsculas, con `filter` y `lambda`: `"juan"` encuentra a *Juan Perez* y a *Juana Garcia*.
Búsqueda por habitación	NUEVA	Devuelve quién ocupa un número de habitación (`buscar_huesped_por_habitacion`).
Submenú de búsquedas	NUEVA	La opción 5 pasó de "Buscar por DNI" a un submenú con los tres criterios.
Reporte de habitaciones libres	NUEVA	Lista ordenada de habitaciones ocupadas y libres usando diferencia e intersección de conjuntos.
Cancelación con `-1`	NUEVA	Cualquier operación puede abandonarse sin modificar el estado. Antes, un error de tipeo obligaba a completar la operación.
Confirmación s/n	NUEVA	`solicitar_confirmacion` acepta *s, si, sí, n, no* y devuelve un booleano; se reutiliza en la configuración de tarifas.
Registro del huésped	MEJORADA	De lista posicional `[nombre, dni, dias, piso, hab, num]` a diccionario con tupla de identidad. Ver capítulo 3.
Reubicación (Swap)	MEJORADA	Una función larga se dividió en `puede_reubicar`, `pedir_huesped_a_reubicar`, `pedir_habitacion_destino` y `mover_huesped`, con una tarea cada una.
Configuración de tarifas	MEJORADA	Se dividió en 5 subfunciones (precio base, pisos de lujo, captura, generación y resumen).
Redimensionar hotel	MEJORADA	Ahora muestra una advertencia de pérdida de datos y permite cancelar antes de borrar.
Check-in con hotel lleno	CORREGIDA	En la Parte 1, si no quedaban habitaciones libres, el pedido de habitación se repetía sin fin. Ahora `hay_habitaciones_libres` lo detecta y avisa.
Validación de rangos repetida	CORREGIDA	La verificación "¿existe esta habitación?" estaba copiada en varias funciones. Se centralizó en `habitacion_existe`.

2.2 Nuevas validaciones (Sección 11)

Situación	Respuesta del sistema	Función
Dato obligatorio vacío	El patrón regex no acepta cadenas vacías; se informa y se vuelve a pedir.	`validar_generico`
Tipo incorrecto (letras en un número)	"Debe ingresar un número entero válido" y nuevo ingreso.	`PATRON_NUMERO_VALIDO`
Valor fuera de rango	Pisos y habitaciones de 1 a 99; piso de lujo entre 2 y el total; precio mayor a 0; días mayor a 0.	`solicitar_dimension_valida`
Opción de menú inexistente	"Opción inválida. Intente nuevamente."	`menu_principal`
DNI ya registrado	Se informa el DNI repetido y se pide otro (o `-1`).	`validar_dni`
Habitación inexistente o no libre	Se informa el motivo y se vuelve a pedir.	`solicitar_habitacion_libre`

Situación	Respuesta del sistema	Función
Restablecer una habitación que no está en limpieza	"La habitación no está en limpieza."	`restablecer_limpieza`
Operar sin huéspedes o sin habitaciones libres	Se avisa antes de pedir datos innecesarios.	`hotel_vacio`, `puede_reubicar`

3. Estructuras de datos

ÍTEM 15.B

Cada estructura se eligió por una propiedad concreta que el dato necesita. La siguiente figura muestra cómo se combinan dentro del registro de un huésped.

```
# huespedes.py · realizar_checkin()
datos_huesped = {                                  # DICCIONARIO: registro con campos por nombre
    "identidad": (dni, nombre),                   # TUPLA: no cambia nunca
    "dias": dias,                                 # int
    "ubicacion": [piso, habitacion, num_comercial]  # LISTA: cambia en el swap
}
lista_huespedes.append(datos_huesped)              # lista de diccionarios
```

3.1 Conjuntos (`set`)

Uso 1 · Evitar DNI duplicados

Dónde: `huespedes.validar_dni`

Se arma un conjunto con los DNI de los huéspedes alojados y se pregunta si el nuevo DNI pertenece.

```
dnis_registrados = set()
for huesped in lista_huespedes:
    dnis_registrados.add(huesped["identidad"][0])
while dni_nuevo in dnis_registrados:
    ...  # informa y pide otro DNI
```

Por qué: un conjunto no admite repetidos y la pertenencia (`in`) se resuelve por *hash*, en tiempo constante promedio, sin recorrer la lista elemento por elemento.

Uso 2 · Comparar información

Dónde: `reportes.reporte_habitaciones_libres`

Se construyen dos conjuntos de números de habitación y se operan:

```
todas    = {101, 102, 103, 201, 202, 203}
ocupadas = {102, 201}
libres   = todas - ocupadas   # diferencia
comunes  = todas & ocupadas   # intersección
```

Por qué: la pregunta "¿qué habitaciones no están ocupadas?" es exactamente una diferencia de conjuntos. La intersección asegura que solo se informen ocupadas que existen en el hotel actual.

3.2 Tuplas (`tuple`)

Qué representa	Dónde se usa	Por qué una tupla
Identidad del huésped `(dni, nombre)`	`realizar_checkin`; se lee en búsquedas, check-out y reportes.	El DNI y el nombre se cargan una vez y no deben modificarse durante la estadía. La tupla es inmutable: un intento de cambiar `identidad[0]` produce un error en lugar de alterar el dato. Además, agrupa dos valores que siempre van juntos.
Coordenadas de habitación `(piso, habitacion)`	`obtener_indices_matriz`, `solicitar_habitacion_libre`	Una función devuelve dos valores relacionados en un solo retorno, que se desempaqueta: `piso, hab = obtener_indices_matriz(201)`.
Destino del swap `(piso, habitacion, numero)`	`pedir_habitacion_destino`, `mover_huesped`	Viaja como un único valor entre funciones; `(-1, -1, -1)` indica cancelación.

> **Por qué la ubicación no es una tupla**
> La ubicación del huésped sí cambia (en el swap se actualizan piso, habitación y número), por eso es una lista. La decisión tupla/lista sigue una regla simple: **lo que no debe cambiar es tupla, lo que puede cambiar es lista**.

3.3 Diccionarios (`dict`)

Qué representa	Dónde se usa	Por qué un diccionario
Registro de un huésped	`realizar_checkin` y todas las funciones de `huespedes.py` y `reportes.py`	Se accede por nombre de campo (`huesped["dias"]`) en lugar de por posición (`huesped[2]`). Agregar un campo nuevo no rompe el código existente, cosa que sí ocurría con las listas posicionales de la Parte 1.

Qué representa	Dónde se usa	Por qué un diccionario
Estado completo del hotel `{"pisos", "habitaciones", "huespedes", "tarifas"}`	`guardar_datos_hotel`, `cargar_datos_hotel`, `menu_principal`	Es la forma natural de un objeto JSON: cada clave se convierte en un campo del archivo y al leerlo se recupera por nombre (`datos["tarifas"]`).

3.4 Otras estructuras del sistema

Estructura	Uso y justificación
Matriz del hotel lista de listas	`hotel[piso][habitacion]` contiene `"L"`, `"O"` o `"S"`. Refleja la forma física del edificio y da acceso directo a cualquier habitación.
Matriz de tarifas `[[piso, precio], ...]`	Una fila por piso con su precio por noche; se genera una vez y se consulta en cada liquidación.
Lista de huéspedes lista de diccionarios	Colección variable que crece en el check-in y se reduce en el check-out; se recorre con `for`, `filter` y `map`.

4. Persistencia

ÍTEM 15.C

La información se guarda exclusivamente en archivos (Sección 12.2). Cada formato cumple un propósito distinto: **JSON** para el estado del hotel y **texto plano** para el historial de facturación.

4.1 Respuesta a las preguntas de la consigna

Pregunta	Respuesta
¿Qué información se almacena?	Las dimensiones del hotel, el esquema de tarifas por piso, los huéspedes alojados (identidad, días y ubicación) y el historial de check-outs realizados.
¿En qué archivo?	`hotel_data.json` (estado del hotel), `facturacion.txt` (bitácora) y `facturacion.csv` (exportación).
¿Qué se guarda en JSON?	El estado actual: `pisos`, `habitaciones`, `huespedes` y `tarifas`. Es información estructurada que se sobrescribe completa en cada guardado.
¿Qué se guarda en texto?	Una línea por cada check-out (fecha, DNI, nombre, habitación, noches y total cobrado). Es información histórica que solo se agrega al final, nunca se reescribe.
¿Cómo se recupera?	El JSON se lee al iniciar y reconstruye el hotel (ver 4.3). La bitácora se lee para mostrar el historial y para generar el CSV (ver 4.4).

4.2 Archivo JSON · `hotel_data.json` IMPLEMENTADO

```
{
    "pisos": 2,
    "habitaciones": 4,
    "huespedes": [
        {
            "identidad": ["40123456", "Juan Perez"],
            "dias": 10,
            "ubicacion": [1, 0, 201]
        }
    ],
    "tarifas": [[1, 10000.0], [2, 12000.0]]
}
```

Cuándo se escribe: al terminar la configuración inicial, después de cada check-in, check-out, swap, restablecimiento y redimensionamiento, y al salir con la opción 0.

Cómo: `json.dump(datos, archivo, ensure_ascii=False, indent=4)` en modo `"w"` y codificación UTF-8. `ensure_ascii=False` conserva tildes y eñes legibles (*Muñoz*, no *Mu\u00f1oz*).

Conversión Python ↔ JSON:

Python	JSON	Al leer
`dict`	objeto	`dict`
`list`	arreglo	`list`
`tuple`	arreglo	`list`
`str` / `int` / `float`	texto / número	igual

Detalle técnico: la tupla en JSON

JSON no tiene tipo tupla, así que `json.dump` guarda `identidad` como arreglo y `json.load` lo devuelve como lista. Como el sistema siempre lee la identidad por índice (`[0]` DNI, `[1]` nombre), el comportamiento es idéntico antes y después de reiniciar. Si se quisiera mantener la inmutabilidad también después de cargar, basta con `huesped["identidad"] = tuple(huesped["identidad"])` al recuperar los datos.

4.3 Recuperación al iniciar

1 · Leer `cargar_datos_hotel()` abre el JSON con `with open(..., "r")` › **2 · ¿Hay datos?** Si `pisos != -1`, se usan los datos guardados › **3 · Reconstruir** `inicializar_hotel(pisos, habs)` crea la matriz en `"L"` › **4 · Ocupar** Por cada huésped: `hotel[p][h] = "O"` según su ubicación › **5 · Operar** El menú arranca con el hotel en el mismo estado

Si el archivo no existe (primera ejecución), `cargar_datos_hotel` devuelve una estructura con `pisos = -1` y `main.py` inicia la configuración guiada, que termina guardando el primer JSON. Así se cumple el ciclo pedido en la Sección 8: generar → guardar → cerrar → volver a ejecutar →

recuperar.

4.4 Archivo de texto · `facturacion.txt` IMPLEMENTADO

> **Implementación completa e integrada**
> La bitácora transaccional se gestiona mediante el módulo `archivos.py` (funciones `inicializar_archivo_txt`, `registrar_factura_txt` y `leer_y_procesar_facturacion_txt`). Cada Check-out registra el cobro automáticamente y la opción 7 del menú permite realizar auditorías históricas acumulando la facturación total.

Operación	Implementación técnica
Inicialización	`inicializar_archivo_txt()` asegura la existencia física de `facturacion.txt` al arrancar el sistema con modo `"a"` y codificación UTF-8.
Escritura	En cada Check-out, `huespedes.py` invoca `registrar_factura_txt(dni, nombre, monto)` en modo agregado (`"a"`). Escribe de inmediato sin leer ni reescribir lo anterior, asentando el comprobante en disco.
Formato	Campos delimitados por `;` : `40123456;Juan Perez;108000.00`
Lectura y auditoría	La opción 7 invoca `leer_y_procesar_facturacion_txt()`, recorriendo el archivo línea a línea con `split(";")`, mostrando cada comprobante y acumulando la recaudación histórica real.
Exportación CSV	La opción 8 invoca `exportar_facturacion_a_csv()`, que procesa el archivo de texto y escribe `facturacion.csv` mediante la librería estándar `csv` (`csv.writer`) con encabezados `["DNI", "Huesped", "Monto_Cobrado"]` para Microsoft Excel o Google Sheets.
Excepciones	Manejo completo con `try / except / else / finally` capturando `FileNotFoundError`, `PermissionError`, `(ValueError, IndexError)` y `OSError`.

Por qué texto y no JSON para el historial: el JSON se reescribe completo en cada guardado; un historial que solo crece se maneja mejor agregando líneas, sin cargar todo en memoria. Además, una bitácora en texto se puede leer directamente con cualquier editor como respaldo de auditoría.

5. Excepciones

ÍTEM 15.D

La Sección 6 pide no usar `try / except` indiscriminadamente. El criterio del equipo fue: **prevenir con validación los errores que dependen del usuario y capturar con excepciones los que dependen del entorno**, es decir, del sistema de archivos.

5.1 Situaciones controladas

Situación	Excepción	Dónde	Cómo se controla
Primera ejecución: el archivo todavía no existe	`FileNotFoundError`	`cargar_datos_hotel`	Informa "El archivo no existe" y devuelve una estructura vacía con `pisos = -1`; el sistema inicia la configuración guiada en lugar de cerrarse.
El sistema operativo no permite leer o escribir el archivo	`PermissionError`	lectura y escritura	Informa "No tiene permisos para acceder al archivo" y el programa sigue funcionando.
El JSON fue editado a mano o quedó dañado	`ValueError`, `IndexError`	`cargar_datos_hotel`	`json.JSONDecodeError` es subclase de `ValueError`, por lo que queda capturada. Se informa "El archivo contiene registros incorrectos".
Un dato no se puede convertir a JSON	`TypeError`, `ValueError`	`guardar_datos_hotel`	Se informa y la función devuelve `guardado = False`.
Disco lleno, ruta inválida u otro error de E/S	`OSError`	lectura y escritura	Se muestra el detalle del error. Se captura después de las específicas porque es su clase padre.

5.2 Estructura completa: `try / except / else / finally`

```
# archivos.py · guardar_datos_hotel()
try:
    archivo = open(nombre_archivo, "w", encoding="UTF-8")
    json.dump(datos, archivo, ensure_ascii=False, indent=4)
    guardado = True
except PermissionError:            # específica primero...
    print("ERROR. No tiene permisos para acceder al archivo.")
except (ValueError, TypeError):
    print("ERROR. El archivo contiene registros incorrectos.")
except OSError as error:           # ...la general al final
    print("ERROR al trabajar con el archivo:", error)
else:                              # solo si no hubo excepción
    print("Escritura finalizada correctamente.")
finally:                           # siempre: cierra el archivo
    if archivo != None:
        archivo.close()
```

Orden de los `except`

`PermissionError` es subclase de `OSError`. Si `OSError` fuera primero, el mensaje específico nunca se mostraría.

`else`

Separa el camino exitoso del código que puede fallar: el mensaje de éxito no se imprime si la escritura falló.

`finally`

Garantiza que el archivo se cierre aunque `json.dump` falle a mitad de la escritura.

5.3 Errores prevenidos sin `try`

Las conversiones como `int(dato)` o `float(dato)` podrían lanzar `ValueError`. En lugar de envolverlas en `try`, el dato se valida antes con una expresión regular en `validar_generico`, que pide reingresar hasta que el formato sea correcto. Cuando se llega a la conversión, el error ya es imposible.

Riesgo	Prevención
`int("abc")` → `ValueError`	`PATRON_NUMERO_VALIDO = r"^\d+$"` solo deja pasar dígitos.
`hotel[5][0]` en un hotel de 2 pisos → `IndexError`	`habitacion_existe(matriz, piso, hab)` verifica los límites antes de acceder.

Riesgo	Prevención
División por cero en el porcentaje de ocupación	`if total_habitaciones > 0` antes de dividir.
Huésped inexistente	Las búsquedas devuelven `[]` o `-1` y se informa al usuario.

6. Pruebas unitarias

ÍTEM 15.E

`tests.py` contiene 10 pruebas sobre **6 funciones distintas** de 4 módulos, con la distribución exigida por la Sección 9. Se ejecutan de forma independiente del programa principal.

4	3	2	1
casos válidos	casos inválidos	casos límite	caso adicional

6.1 Funciones testeadas y casos considerados

#	Tipo	Función	Datos	Esperado
01	VÁLIDO	`hotel.inicializar_hotel`	2 pisos, 3 habitaciones	matriz 2×3 de `"L"`
02	VÁLIDO	`tarifas.obtener_tarifa_habitacion`	piso 2; tarifas de 3 pisos	`12000.0`
03	VÁLIDO	`reportes.calcular_subtotal`	5 noches, piso 1 a $10.000	`50000.0`
04	VÁLIDO	`reportes.calcular_subtotal`	10 noches, piso 1 a $10.000	`90000.0`
05	INVÁLIDO	Detección con `set` (lógica de `validar_dni`)	DNI `11111111` ya registrado	`True` (duplicado)
06	INVÁLIDO	`hotel.habitacion_existe`	piso 5 en un hotel 2×2	`False`
07	INVÁLIDO	`huespedes.buscar_huesped_por_habitacion`	habitación 999	`[]`
08	LÍMITE	`reportes.calcular_subtotal`	exactamente 7 noches	`70000.0`
09	LÍMITE	`hotel.inicializar_hotel`	1 piso, 1 habitación	`[["L"]]`
10	ADICIONAL	`huespedes.buscar_huespedes_por_nombre`	`"juan"` entre 3 huéspedes	Juan Perez y Juana Garcia

6.2 Ejecución y salida

Opción 1 · Python puro (no requiere instalar nada). Muestra los 5 datos exigidos por prueba:

```
$ python tests.py
Prueba [04] – CASO VÁLIDO
  1. Función:   reportes.calcular_subtotal
  2. Datos:     huesped={'dias': 10, ...}
  3. Esperado:  90000.0
  4. Obtenido:  90000.0
  5. Resultado: [APROBADO]
...
RESUMEN FINAL: 10/10 PRUEBAS APROBADAS
```

Opción 2 · PyTest (autorizado por la cátedra). Descubre las funciones `test_*` y evalúa los `assert`:

```
$ python -m pytest tests.py -v
test_01_inicializacion_matriz_valida  PASSED
test_02_obtener_tarifa_piso_existente PASSED
...
test_10_busqueda_parcial_nombre_...   PASSED
========== 10 passed in 0.08s ==========
```

Cada prueba arma sus propios datos (no lee el JSON ni pide `input`), por eso el resultado no depende del estado del hotel ni del orden de ejecución.

6.3 Guía para la defensa

La Sección 9 indica que el docente puede elegir cualquier prueba y preguntar qué comprueba, por qué se eligió, qué se espera y qué pasaría si cambiaran los datos.

#	Qué comprueba y por qué se eligió	Resultado esperado	Qué pasaría si se modifican los datos
01	Que la matriz tenga tantas filas como pisos y columnas como habitaciones, todas libres. Es la base de todo el sistema.	`[["L","L","L"], ["L","L","L"]]`	Con `(3, 2)` la matriz sería 3×2 y la prueba fallaría contra el esperado 2×3: confirma que no se invierten pisos y habitaciones.
02	Que se devuelva el precio del piso pedido y no el de otro. Es el dato que alimenta toda la facturación.	`12000.0`	Con piso 3 devolvería `15000.0`. Con un piso inexistente (p. ej. 9) la función devuelve el precio del último piso como respaldo.
03	El cálculo básico `precio × noches` sin descuento.	`50000.0`	Con 6 noches: `60000.0`. Con el huésped en el piso 2 (`ubicacion[0] = 1`) se usaría $12.000 → `60000.0`.
04	Que la `lambda` aplique el 10 % cuando hay más de 7 noches.	`90000.0` (100.000 − 10 %)	Si se quitara el descuento del código, el obtenido sería `100000.0` y la prueba lo detectaría.
05	Que un DNI repetido se detecte por pertenencia al conjunto. Es la regla que impide registrar dos veces a la misma persona.	`True`	Con un DNI nuevo (`"22222222"`) el resultado sería `False`, es decir, se permitiría el registro.
06	Que una coordenada fuera de la matriz se rechace antes de acceder, evitando un `IndexError`.	`False`	Con piso 1 y habitación 1 devolvería `True`. Con valores negativos también devuelve `False`.
07	Que buscar en una habitación sin huésped devuelva una lista vacía y no un error.	`[]`	Con la habitación 101 devolvería una lista con el registro de *Ana*.
08	La frontera exacta de la regla "más de 7": con 7 todavía no hay descuento. Detecta si alguien cambia `>` por `>=`.	`70000.0`	Con 8 noches: `72000.0` (80.000 − 10 %). Si el código usara `>=`, con 7 daría `63000.0` y la prueba fallaría.
09	El hotel más chico permitido (1×1): los bucles deben funcionar con una sola iteración.	`[["L"]]`	Con `(0, 0)` devolvería `[]`; ese caso no llega a la función porque la validación exige un mínimo de 1.
10	Búsqueda parcial e insensible a mayúsculas, el caso real de un recepcionista que no recuerda el nombre completo.	`[h1, h3]`	Con `"JUAN"` el resultado es el mismo (se compara con `.lower()`). Con `"lopez"` devolvería solo `[h2]`.

7. Cambios respecto de la Parte 1

ÍTEM 15.F

Qué corresponde a la primera versión y qué se incorporó o modificó en esta segunda etapa.

7.1 Comparación por aspecto

Aspecto	Parte 1	Parte 2
Organización	Un único archivo `main.py` con toda la lógica.	7 módulos con una responsabilidad cada uno; `main.py` solo coordina.
Registro del huésped	Lista posicional `[nombre, dni, dias, piso, hab, num]`.	Diccionario con tupla `identidad` y lista `ubicacion`.
Unicidad de DNI	No se controlaba.	Conjunto en `validar_dni`.
Habitaciones libres	Solo se veían en la grilla.	Reporte por diferencia de conjuntos.
Búsquedas	Por DNI (menú) y filtro por piso.	Submenú: DNI, nombre parcial y habitación.
Persistencia	Ninguna: los datos se perdían al salir.	Triple persistencia: JSON (estado), TXT (bitácora continua en modo append) y CSV (exportación tabular).
Opciones de menú	8 opciones básicas.	10 opciones estructuradas con auditoría histórica y exportación a planillas.
Errores	Solo validaciones con regex.	Regex más `try / except / else / finally` en archivos.
Cancelación	No existía.	`-1` en todas las entradas.
Hotel completo	El check-in quedaba pidiendo habitación sin fin.	Se detecta y se avisa.
Redimensionar	Borraba todo sin advertencia.	Advertencia y opción de cancelar.
Pruebas	Manuales, ejecutando el programa.	10 pruebas automáticas en `tests.py`.

7.2 Inventario de funcionalidades

Se conserva de la Parte 1

- Matriz del hotel y estados `L` / `O` / `S`.
- Numeración comercial (101, 102, ...) y conversión inversa.
- Grilla visual por piso.
- Check-in, check-out con liquidación, swap y restablecer limpieza.
- Tarifas por piso con pisos de lujo.
- Descuento del 10 % con `lambda` para más de 7 noches.
- Reporte de ocupación y recaudación con `map` y `reduce`.
- Validación con expresiones regulares.

Se incorpora o modifica en la Parte 2

- NUEVA Persistencia JSON y recuperación.
- NUEVA Conjuntos: DNI único y habitaciones libres.
- NUEVA Búsqueda por nombre y por habitación.
- NUEVA Manejo de excepciones.
- NUEVA Pruebas unitarias.
- NUEVA Cancelación con `-1` y confirmación s/n.
- MEJORADA Huésped como diccionario con tupla.
- MEJORADA Swap y tarifas divididos en subfunciones.
- CORREGIDA Bucle con hotel completo.
- REEMPLAZADA El filtro por piso se sustituyó por las búsquedas por nombre y habitación.

8. Cumplimiento de las restricciones técnicas

SECCIÓN 12

Restricción	Cómo se respeta	Estado
12.1 · Sin clases	No se usa `class`, objetos propios, herencia, métodos de clase ni constructores. El sistema está hecho con funciones, parámetros y valores de retorno.	CUMPLE
12.2 · Sin bases de datos	No se usa ningún motor ni ORM. La persistencia es solo con archivos JSON y de texto.	CUMPLE
12.3 · Solo librerías vistas	Sistema: `re` (validaciones), `json` (persistencia) y `functools` (`reduce`). Exportación: `csv`. Pruebas: `pytest`, autorizado por la cátedra y opcional, porque `tests.py` también corre con Python puro.	CUMPLE
12.4 · Código explicable	Sin decoradores, generadores, comprensiones anidadas ni construcciones fuera del programa. Los bucles usan variables de control booleanas en lugar de `while True` con `break`.	CUMPLE

Uso de programación funcional

Herramienta	Dónde	Para qué
`lambda`	`calcular_subtotal`	Regla de descuento: `subtotal * 0.9 if dias > 7 else subtotal`.
`filter`	`buscar_huespedes_por_nombre`, `buscar_huesped_por_habitacion`	Quedarse con los huéspedes que cumplen un criterio.
`map`	`calcular_recaudacion_total`, `generar_reporte_ocupacion`	Transformar cada huésped en su monto y cada piso en su cantidad de ocupadas.
`reduce`	`generar_reporte_ocupacion` (×3), `calcular_recaudacion_total`	Acumular total de habitaciones, ocupadas y recaudación.

Anexo A · Arquitectura y módulos

SECCIÓN 10

Las flechas indican "importa funciones de". No hay dependencias circulares y la información se comparte por parámetros y valores de retorno.

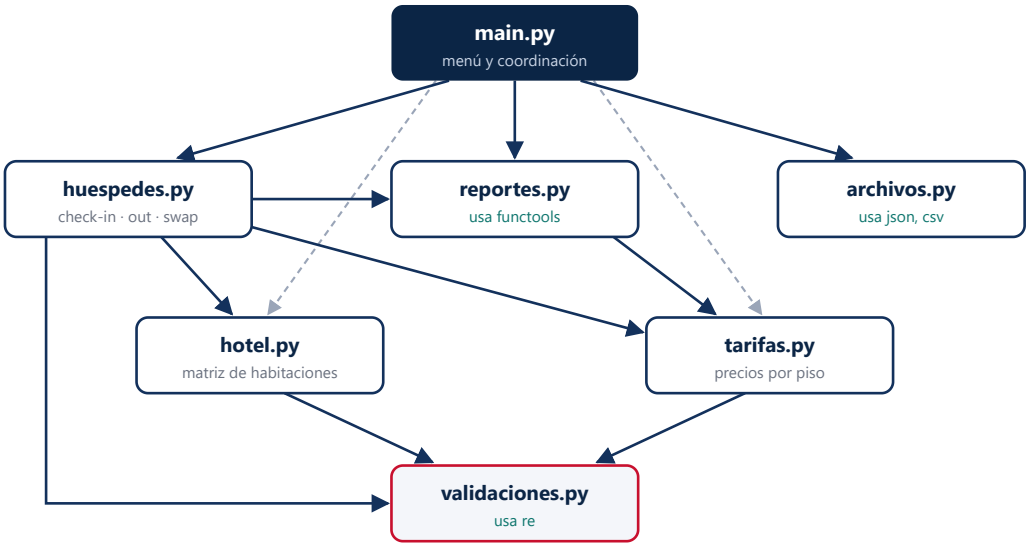


Figura A.1 · Dependencias entre módulos. Las líneas punteadas son importaciones directas de `main.py` a `hotel.py` y `tarifas.py`; `main.py` también importa `validaciones.py` (no dibujado para mayor claridad).

Módulo	Funciones	Responsabilidad	Funciones principales
`main.py`	2	Cargar datos, mostrar menús, delegar cada opción y guardar.	`menu_principal`, `menu_busquedas`
`hotel.py`	9	Crear y mostrar la matriz; convertir coordenadas; verificar existencia y disponibilidad.	`inicializar_hotel`, `renderizar_hotel`, `habitacion_existe`
`huespedes.py`	14	Todas las operaciones sobre huéspedes.	`realizar_checkin`, `realizar_checkout`, `reubicar_huesped_swap`
`tarifas.py`	7	Configurar, mostrar y consultar precios.	`configurar_tarifas_hotel`, `obtener_tarifa_habitacion`
`reportes.py`	4	Cálculos e indicadores con programación funcional y conjuntos.	`calcular_subtotal`, `calcular_recaudacion_total`
`validaciones.py`	3	Patrones, mensajes de error y pedido de datos con reintento.	`validar_generico`, `solicitar_dimension_valida`
`archivos.py`	6	Persistencia JSON, auditoría TXT y exportación CSV con excepciones.	`cargar_datos_hotel`, `guardar_datos_hotel`, `registrar_factura_txt`, `leer_y_procesar_facturacion_txt`, `exportar_facturacion_a_csv`
`tests.py`	12	10 pruebas, formateador de salida y ejecutor.	`test_01` … `test_10`, `ejecutar_suite`

Anexo B · Ciclo de vida de una habitación

SECCIÓN 13

Cada celda de la matriz es un pequeño autómata con tres estados. Solo las operaciones del menú pueden cambiarlo, y cada cambio se guarda en el JSON.

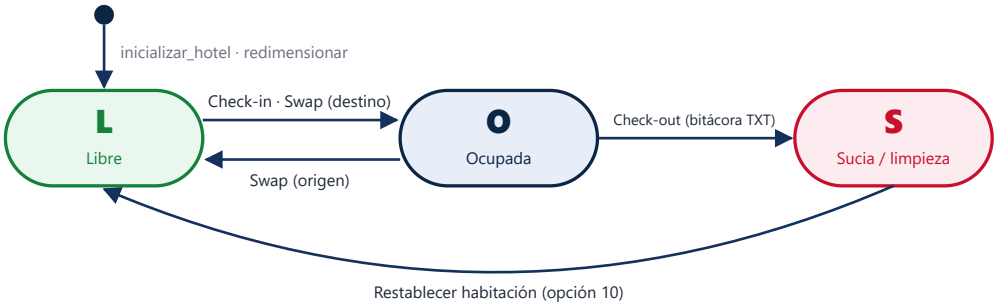


Figura B.1 · Transiciones de estado de una habitación.

Transición	Función	Condición previa validada
L → O (check-in)	`realizar_checkin`	La habitación existe y está libre; DNI válido y no repetido; días > 0.
O → S (check-out)	`realizar_checkout`	Hay huéspedes y el DNI corresponde a un huésped alojado.
O → L / L → O (swap)	`mover_huesped`	Hay huéspedes, hay habitaciones libres y el destino existe y está libre.
S → L (restablecer)	`restablecer_limpieza`	La habitación existe y está en estado `S`.

> **Por qué existe el estado S**
> En un hotel real, una habitación recién desocupada no se puede entregar hasta limpiarla. El estado intermedio impide que el check-in la ofrezca como libre y obliga a una confirmación explícita.

Anexo C · Instrucciones de ejecución

SECCIÓN 16

Pasos para ejecutar el sistema en otro equipo.

Requisitos

- Python 3.8 o superior (probado en Python 3.14 sobre Windows).
- No se necesitan librerías externas para el sistema. `pytest` es opcional.

Pasos

```
# 1. Obtener el proyecto
git clone https://github.com/lucassebastianramos/room-master.git
cd room-master

# 2. Ejecutar el sistema
python main.py

# 3. Ejecutar las pruebas sin instalar nada
python tests.py

# 4. (Opcional) Ejecutar las pruebas con PyTest
python -m pip install pytest
python -m pytest tests.py -v
```

> **Nota para Windows**
> Si el comando `pytest` no se reconoce, usar `python -m pytest`. Ocurre porque `pip` instala el ejecutable en una carpeta que no está en el `PATH`.

Archivos que genera el sistema

Archivo	Contenido	Se crea
`hotel_data.json`	Estado del hotel: dimensiones, tarifas y huéspedes.	Al terminar la configuración inicial
`facturacion.txt`	Bitácora de check-outs.	En el primer check-out
`facturacion.csv`	Exportación del historial para planillas de cálculo.	Al exportar

Primer uso sugerido

1. Configurar un hotel de 3 pisos × 4 habitaciones, base $10.000 y lujo desde el piso 3 con 30 %.
2. Opción 2: registrar dos huéspedes. Intentar repetir un DNI para ver la validación con conjuntos.
3. Opción 6: ver ocupación, habitaciones libres y recaudación proyectada.
4. Opción 0: salir. Volver a ejecutar `python main.py` y verificar con la opción 1 que el hotel se recuperó.
5. Opción 3: hacer check-out (se calcula el total y se asienta en `facturacion.txt`); la habitación queda en `S`.
6. Opción 7: auditar el historial de facturación acumulado desde el archivo TXT.
7. Opción 8: exportar el historial a `facturacion.csv` para abrirlo en Excel.
8. Opción 10: restablecer la habitación en limpieza a `L`.